

Хеш-таблицы

Почему `dict` работает за $O(1)$ и почему он помнит порядок вставки?

Вопрос дня

```
>>> d = {"one": 1, "two": 2}
>>> d["two"]
2
>>> big = {str(i): i for i in range(10**6)}
>>> big["999999"]
999999
>>> list(d)
['one', 'two']
>>> del d["one"]; d["one"] = 9
>>> list(d)
['two', 'one']
>>> {[1, 2]: 0}
TypeError: unhashable type: 'list'
```

Поиск в словаре из двух ключей и в словаре из миллиона стоит одинаково, и перебора при этом нет.

Вопрос дня

```
>>> d = {"one": 1, "two": 2}
>>> d["two"]
2
>>> big = {str(i): i for i in range(10**6)}
>>> big["999999"]
999999
>>> list(d)
['one', 'two']
>>> del d["one"]; d["one"] = 9
>>> list(d)
['two', 'one']
>>> {[1, 2]: 0}
TypeError: unhashable type: 'list'
```

Поиск в словаре из двух ключей и в словаре из миллиона стоит одинаково, и перебора при этом нет.

Порядок ключей совпадает с порядком вставки, хотя никто его не запоминал.

Список ключом быть не может, а строка может.

Сегодня разбирается, что лежит в памяти, чтобы объяснить все три наблюдения.

РАЗДЕЛ 1

Хеш

Как вообще искать значение по ключу

способ	поиск	что требуется от ключа
массив пар, перебор	$O(n)$	сравнение на равенство
отсортированный массив	$O(\log n)$	сравнение на «меньше»
массив, ключ — индекс	$O(1)$	ключ — целое от 0 до n

Третья строка — обычный массив: `a[i]` находится за фиксированное время при любом размере. Это известно с первого дня, и цена — требование к ключу: маленькое целое число.

Главное

Хеш-таблица берёт третью строку и снимает её требование. Ключ превращается в число функцией, а число — в индекс остатком.

Хеш-функция

```
uint32_t fnv1a(const char *s)
{
    uint32_t h = 2166136261u;
    for (const unsigned char *p = (const unsigned char *)s; *p; p++) {
        h ^= *p;           // смешали очередной байт
        h *= 16777619u;    // размешали по всем 32 битам
    }
    return h;
}
```

Функция читает байты и возвращает число. Требований к ней два:

- равные ключи дают равные числа — без этого таблица теряет ключи;
- разные ключи дают разные числа настолько часто, насколько возможно.

Первое — обязательно, второе — вопрос качества. Умножение здесь переполняется почти на каждом шаге: для `unsigned` это определённая операция, и результат — младшие 32 бита произведения.

Хеш, который выполняет только первое требование

```
uint32_t sum_hash(const char *s) // сумма байтов
```

анаграммы	сумма	FNV-1a
listen, silent abc, bca, cab	совпали 294 у всех трёх	разошлись три разных числа

200 ключей key000 ..., 64 корзины	сумма	FNV-1a
разных значений хеша	20	200
пустых корзин	44	0
ключей в самой длинной корзине	19	4

Сумма байтов — законная хеш-функция: равные строки она переводит в равные числа. Перестановку букв она не замечает.

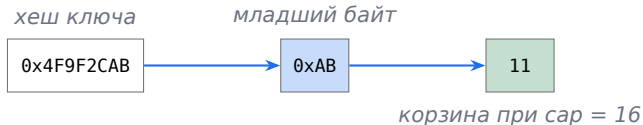
Главное

Качество хеша не влияет на правильность и полностью определяет скорость.

Из хеша в номер корзины

```
size_t bucket = hash % cap;           // работает при любом cap  
size_t bucket = hash & (cap - 1);    // только если cap — степень двойки
```

Вторая строка делает то же самое, что первая, и не содержит деления. Цена — требование к `cap`; как следствие, на корзину влияют только младшие биты.



Отсюда второе требование к хеш-функции: каждый байт ключа обязан влиять на младшие биты результата.

РАЗДЕЛ 2

Коллизии

Коллизии неизбежны

Ключей бесконечно много, корзин cap . Значит, найдутся два ключа с одной корзиной — это верно для любой хеш-функции.

Вопрос в том, когда это случится на практике:

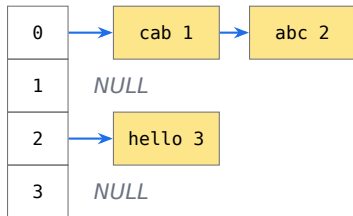
cap	первая коллизия на элементе	$\sqrt{\text{cap}}$	занято таблицы
1 024	37.6	32	3.7 %
65 536	308.9	256	0.5 %
1 048 576	1 229.0	1 024	0.1 %

Случайные ключи, среднее по двумстам опытам. Первая коллизия случается около $\sqrt{\text{cap}}$ -го ключа — при заполненности в доли процента.

Главное

Коллизия — не редкий случай, а обычный режим работы. Основа хеш-таблиц — это в первую очередь способ обработки коллизий.

Способ первый: цепочки



Корзина — голова односвязного списка. Ключей в таблице может быть сколько угодно; при n ключах и cap корзинах средняя длина цепочки равна $\alpha = n/cap$.

корзин	α	сравнений
2 048	0.49	1.27
512	1.95	1.86
128	7.81	4.56
8	125.00	63.00

Тысяча ключей и поиск по ним. Число сравнений растёт как $1 + \alpha/2$: при фиксированном количестве корзин таблица превращается по асимптотике в связный список.

Чтение по ключу: цепочки

```
struct Node *lookup(struct Node **buckets, size_t cap, const char *key)
{
    for (struct Node *n = buckets[fnv1a(key) % cap]; n; n = n->next)
        if (strcmp(n->key, key) == 0)    // корзина общая, ключи — нет
            return n;
    return NULL;                          // конец цепочки: ключа нет
}
```

Хеш и обращение к корзине стоят одинаково при любом числе ключей. Всё остальное время уходит на проход цепочки, и сравнение строк нужно на каждом узле: общая корзина означает только совпадение остатка.

Найденный ключ обходит цепочку в среднем наполовину, отсутствующий — до конца. Тысяча ключей в 128 корзинах: 4.56 сравнения на найденный ключ, 7.62 на отсутствующий при средней длине цепочки 7.81.

Способ второй: открытая адресация

Ключ лежит прямо в таблице. Если корзина занята, берётся следующий слот по кругу, и так до первого свободного.

pink 1	brown 2	olive 4	gold 3	black 7	.	white 5	.	.	.	cyan 10	gray 6	red 8	green 11	blue 9	lime 12
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Двенадцать ключей в таблице на шестнадцать слотов. Ключ `olive` попал в корзину 12, а лёг в слот 2: слоты 12, 13, 14, 15, 0 и 1 были заняты, и просмотреть пришлось семь слотов. Занятые слоты собираются в сплошные участки, и каждый ключ, попавший в участок, удлиняет его.

Главное

Ключей не может быть больше, чем слотов. Зато вся таблица — один блок памяти: ни узлов, ни указателей, ни отдельного `malloc` на ключ.

Чтение по ключу: линейное пробирование

```
long find(char *slots[], size_t cap, const char *key)
{
    size_t i = fnv1a(key) % cap;
    for (size_t seen = 0; seen < cap; seen++) {
        if (slots[i] == NULL) return -1;           // пусто: ключа нет
        if (strcmp(slots[i], key) == 0) return (long)i; // нашли
        i = (i + 1) % cap;                         // занят чужим ключом
    }
    return -1;                                     // круг замкнулся
}
```

Остановка на пустом слоте законна: вставка кладёт ключ в первый свободный слот от его корзины, поэтому пустых слотов между ними не остаётся. Счётчик `seen` ограничивает проход одним кругом.

Ключ `olive` с предыдущего кадра находится на седьмом сравнении: слоты 12, 13, 14, 15, 0 и 1 заняты чужими, сам он лежит в слоте 2.

Удаление ломает поиск

Проверка на пустой слот верна, пока между корзиной и ключом нет пустых слотов. Средний ключ из трёх удалён очисткой слота:

.	.	.	word4 4	.	word24 24	.	.
---	---	---	---------	---	-----------	---	---

поиск останавливается здесь

Ключ `word24` лежит в таблице и не находится. Вместо очистки в слот кладут **надгробие** — значение, не равное ни одному ключу:

.	.	.	word4 4	X	word24 24	.	.
---	---	---	---------	---	-----------	---	---

Главное

Пустой слот заканчивает поиск. Надгробие — не заканчивает, но годится для вставки. Из этих двух правил следует всё поведение удаления.

Коэффициент заполнения

α — доля занятых слотов; надгробия считаются занятыми.

α	слотов на поиск	слотов на вставку
0.50	1.50	2.46
0.75	2.51	8.58
0.90	5.57	51.79
0.99	47.24	3 351.42

65 536 слотов, линейное пробирование. Теория: $\frac{1}{2}(1 + \frac{1}{1-\alpha})$ на поиск, $\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$ на вставку.

Главное

Формула растёт неограниченно, но таблица конечна: вставка стоит не больше её размера — в последней строке против 5 000 по формуле — 3 351. Плохой случай — $O(m)$ вместо фиксированного времени, и таблица растёт заранее.

Рост таблицы

Номер корзины зависит от `cap`, поэтому увеличить таблицу простым `realloc` нельзя: все ключи придётся разложить заново.

```
49152 ключей – таблица  65536 -> 131072, переложено 49152
98304 ключей – таблица 131072 -> 262144, переложено 98304
```

```
вставлено ключей:      100000
переездов:              16
переложено при переездах: 196602
```

Перестроек — логарифм от числа ключей, переложено — меньше двух на ключ. Та же амортизация, что у вектора третьего дня:

$$1 + 2 + 4 + \dots + n < 2n.$$

Главное

Отдельная вставка может стоить $O(n)$, но любые n вставок перекладывают меньше $2n$ ключей, каковы бы ни были ключи.

Когда $O(1)$ перестаёт работать

Пять тысяч ключей — перестановки одних и тех же десяти букв.
Таблица с цепочками на 4096 корзин, отличается только хеш-функция:

хеш	занятых корзин	длиннейшая	сравнений	время
сумма байтов	1	5 000	12 497 500	28.3 мс
FNV-1a	1 875	9	6 182	0.2 мс

Первая строка — связный список с лишним шагом: вставка n ключей стоит $O(n^2)$. Это законное поведение хеш-таблицы, она обещает фиксированное время **в среднем по ключам**.

Как это выглядит в Python

Ключи выбирает тот, кто присылает данные, поэтому `hash` для строк в Python солится случайным числом при запуске: одинаковые строки в одном запуске дают одинаковый хеш, в разных — разный.

РАЗДЕЛ 3

Программа из нескольких файлов

Зачем разрезать программу

До сих пор каждая задача была одним файлом. У словаря файлов будет три, и причины к тому три:

- **переиспользование**: тот же словарь понадобится второй программе, и копировать его туда не нужно;
- **границы**: вектор ничего не знает про хеши, словарь ничего не знает про разбор команд, и сломать соседа случайно нельзя;
- **время сборки**: изменился один файл — перекомпилируется один файл.

Единица трансляции

Компилятор обрабатывает по одному файлу `.c` за раз и ничего не знает про остальные. Файл вместе со всем, что подставил в него препроцессор, называется единицей трансляции; на выходе получается объектный файл `.o`.

Два шага сборки

```
$ gcc -c main.c -o main.o      # компиляция: текст -> машинный код
$ gcc -c area.c -o area.o
$ gcc main.o area.o -o prog    # линковка: объектные файлы -> программа
```

```
$ nm main.o
0000000000000000 T main
                 U printf
                 U rect_area
```

```
$ nm area.o
0000000000000000 T rect_area
```

U — имя использовано, но не определено здесь. T — имя определено в этом файле. Линковщик соединяет одно с другим; если пары не нашлось, получается знакомое сообщение:

```
undefined reference to `rect_area'
```

Объявление и определение

area.h — объявления

```
int rect_area(int w, int h);
```

area.c — определение

```
#include "area.h"
int rect_area(int w, int h)
{
    return w * h;
}
```

Объявление сообщает компилятору имя и типы; определение выделяет память или задаёт тело. Объявлений может быть много, определение — ровно одно на всю программу.

Заголовок включают в оба файла: и туда, где функция вызывается, и туда, где она определена. Второе нужно, чтобы компилятор сверил объявление с определением.

Что кладут в заголовок

в заголовке	в .c
объявления функций определения типов (<code>struct</code> , <code>typedef</code>) <code>extern</code> для общих переменных макросы и константы	тела функций переменные уровня файла <code>static</code> для внутренних

Определение функции или переменной в заголовке, включённом в два файла, даёт два определения одного имени, и линковщик такую программу не соберёт:

```
multiple definition of `total'; main.o:(.bss+0x0): first defined here
```

Исключение — `static` и `inline`, но пока проще считать, что определений в заголовке не бывает.

static и extern

```
/* util.c */
static int doubled(int x)
{
    return 2 * x;
}
int quadrupled(int x)
{
    return doubled(doubled(x));
}
```

```
$ nm util.o
0000000000000000 t doubled
0000000000000012 T quadrupled
```

Строчная `t` — имя видно только внутри своего файла. Вызов `doubled` из другого файла не соберётся.

`static` прячет имя от остальной программы: так помечается всё, что не предназначено соседям. Иначе два файла с одинаковым именем функции не соберутся вместе.

Обратное слово — `extern`: переменная определена в другом файле. Так устроены счётчики в `vector.h`.

Страж включения

Препроцессор подставляет текст заголовка вместо `#include`. Если заголовок попадёт в файл дважды — напрямую и через другой заголовок, — компилятор увидит два определения одной структуры:

```
point.h:2:8: error: redefinition of 'struct Point'
```

```
#ifndef POINT_H      /* если POINT_H ещё не определён */
#define POINT_H      /* определить его */

struct Point { int x; int y; };

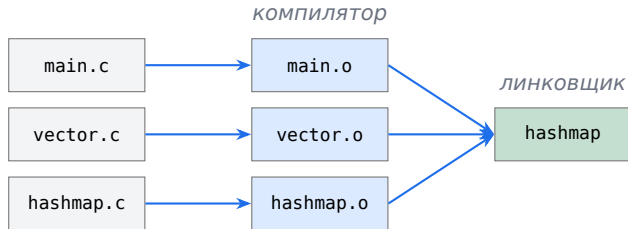
#endif
```

При втором включении `POINT_H` уже определён, и препроцессор выбрасывает всё до `#endif`. Три строки, которые есть в каждом заголовке этого курса и почти в каждом заголовке вообще.

Как это собирается

```
BIN := hashmap  
SRC := main.c vector.c hashmap.c  
  
include ../../../../common/task.mk
```

Две строки перед `include` в `Makefile` каталога задачи, и `make` собирает из трёх файлов одну программу теми же флагами. Без них `make` собрал бы три отдельные программы и не нашёл бы `main` в двух из них.



РАЗДЕЛ 4

Ответ на вопрос дня

Словарь, в котором запись лежит в корзине

```
struct Entry { char *key; uint32_t hash; long value; };    // 24 байта
struct Entry slots[cap];                                // корзина – запись
```

Так словарь устроен в большинстве учебников и так он был устроен в CPython до версии 3.6. Отсюда два следствия:

- заполненность колеблется между третьей и двумя третями, поэтому на пустые слоты уходит от трети до двух третей памяти; при миллионе ключей это 26 мегабайт из 50;
- обход идёт по слотам, то есть в порядке хешей. Он выглядит случайным и меняется при каждой перестройке.

Как это выглядит в Python

До версии 3.6 порядок ключей в словаре был именно таким, и полагаться на него было нельзя. Функция `collections.OrderedDict` существовала как раз потому, что обычный словарь порядка не помнил.

Словарь, в котором корзина хранит номер записи

корзины

1	6	-1	-1	-1	5	-1	4	2	3	7	-1	-1	-1	-1	0
---	---	----	----	----	---	----	---	---	---	---	----	----	----	----	---

записи

apple 1	banana 2	cherry 3	date 4	elder 5	fig 6	grape 7	kiwi 8
---------	----------	----------	--------	---------	-------	---------	--------

В корзине лежит номер записи — четыре байта вместо двадцати четырёх. Сами записи никуда не делись: подешевел пустой слот. -1 — пусто, -2 — надгробие.

памяти под таблицу, байт			
ключей	корзин	запись в корзине	номер в корзине
8	16	384	256
1 000	2 048	49 152	32 192
1 000 000	2 097 152	50 331 648	32 388 608

Откуда берётся порядок вставки

Обход словаря идёт по массиву записей, а он заполняется только с конца. Ни одного дополнительного поля для этого не нужно.

```
>>> d = {"a": 1, "b": 2, "c": 3}
>>> del d["b"]
>>> d["b"] = 4
>>> list(d)
['a', 'c', 'b']
```

Удаление освобождает ключ, оставляет в записи `NULL` и кладёт в корзину надгробие. Дыра в массиве записей остаётся до ближайшей перестройки, которая сдвигает живые записи к началу, сохраняя порядок. Вернувшийся ключ — новая запись в конце.

Главное

Порядок вставки в Python — не обещание разработчиков, а побочный результат экономии памяти. В версии 3.6 он был деталью реализации, с 3.7 — частью языка.

Почему ключ обязан быть неизменяемым

```
>>> {[1, 2]: 0}
TypeError: unhashable type: 'list'
>>> {(1, 2): 0}
{(1, 2): 0}
```

Хеш ключа посчитан один раз, при вставке, и лежит рядом с записью. Если байты ключа изменятся, хеш останется прежним, а корзина станет другой, и ключ потеряется в собственном словаре.

Главное

Запрет на изменение строк из четвёртого дня — то же самое требование. Список изменяем, поэтому хеша у него нет; кортеж неизменяем, поэтому есть.

В С за этим следит программист: словарь копирует ключ себе, и адрес копии известен только ему.

Во что обходятся операции

Таблица третьего дня, к которой была обещана вторая половина:

	ваша реализация	в Python
<i>вектор, третий день</i>		
чтение по индексу	$O(1)$	<code>a[i]</code> — $O(1)$
добавление в конец	амортизированная $O(1)$	<code>append</code> — та же
вставка в начало	$O(n)$	<code>insert(0, x)</code> — та же
поиск значения	$O(n)$	<code>x in a</code> — та же
<i>словарь, сегодня</i>		
поиск по ключу	$O(1)$ в среднем	<code>d[k]</code> — та же
проверка наличия ключа	$O(1)$ в среднем	<code>k in d</code> — та же
вставка нового ключа	амортизированная $O(1)$ в среднем	<code>d[k] = v</code> — та же
замена и удаление	$O(1)$ в среднем	<code>del d[k]</code> — та же
обход	$O(n)$, порядок вставки	<code>for k in d</code> — тот же

Столбцы снова совпадают, а разница между `x in a` и `k in d` — вся причина, по которой словарь существует отдельно от списка.

Почему поиск в словаре не зависит от числа ключей

Ключ превращается в число хеш-функцией, число — в номер корзины маской. Далее просматривается несколько соседних слотов: сколько именно, зависит не от числа ключей, а от заполненности таблицы, которую словарь держит ниже двух третей, перестраиваясь при росте. Отсюда фиксированное время в среднем — и $O(n)$ в худшем случае, когда все ключи попали в одну корзину.

Почему словарь помнит порядок вставки

Корзины хранят не записи, а их номера в отдельном массиве, заполняемом только с конца: по нему идёт обход, и порядок вставки получается бесплатно. Пустой слот при этом стоит четыре байта вместо двадцати четырёх, и таблица меньше примерно на треть.

Итог дня

- Хеш-функция переводит ключ в число. Равные ключи обязаны давать равные числа; всё остальное — вопрос качества, от которого зависит только скорость.
- Номер корзины — остаток от числа корзин, а при степени двойки — маска. Коллизии неизбежны и случаются задолго до заполнения таблицы.
- Цепочки хранят столкнувшиеся ключи в списках, открытая адресация — в соседних слотах. Во втором случае удаление обязано оставлять надгробие.
- Время поиска определяется заполненностью, а не числом ключей. Таблица перестраивается заранее, и перестройка амортизируется, как удвоение вектора.
- Программа из нескольких файлов: объявления в `.h`, определения в `.c`, `static` прячет имя, страж включения защищает от второго включения, линковщик соединяет `U` с `T`.
- Словарь в CPython хранит в корзинах номера записей, а сами записи — подряд и в порядке вставки. Отсюда и экономия памяти, и порядок.

Задачи дня: `day5-hashmap/tasks/`

Вопросы к защите открыты: `defense/questions.md` — посмотрите их до того, как начнёте решать.